Modeling: Home Credit Default Risk

IS 6812 MSBA Capstone 1

AUTHOR
Marco Rodriguez

PUBLISHED
October 11, 2026

1 Introduction

1.1 Business problem

Home Credit lends to customers with little or no formal credit history. Because these applicants lack a conventional bureau score, approval decisions carry avoidable error in both directions: approving applicants who cannot repay creates credit losses, and declining applicants who would have repaid gives up profit and works against the company's mission of expanding credit access.

1.2 Analytic problem

The analytic task is **supervised binary classification**: use information available at application time to estimate each applicant's probability of payment difficulty (`TARGET = 1`). The model's output is a ranked risk score; the business then chooses an approval cutoff based on the cost of each type of mistake. Because only about 8% of applicants default, models are compared on **AUC** (how well they rank risky applicants above safe ones), not on accuracy.

1.3 Purpose of this notebook

This notebook:

1. Establishes a benchmark that any useful model must beat.
2. Compares candidate model types with 3-fold cross-validation.
3. Tests strategies for the 8% class imbalance.
4. Tunes the best model with randomized search.
5. Trains the final model, generates a Kaggle submission, and records the score.
6. Translates the model into business terms: what approval cutoff it supports and what that would mean for default rates and profit.

2 Data preparation

2.1 Applying the data preparation script

All cleaning and feature engineering comes from `data_preparation.py`, built in the previous assignment. It learns every data-dependent value (caps, medians, category levels, bin cutoffs) from the training data only and applies the same values to the test data.

```
# Libraries for data handling, modeling, and evaluation
import re, sys, time
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
from scipy.stats import loguniform, randint, uniform
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict,
        RandomizedSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.dummy import DummyClassifier
from sklearn.metrics import roc_auc_score, accuracy_score
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import RandomOverSampler, SMOTE
import lightgbm as lgb
from lightgbm import LGBMClassifier
from xgboost import XGBClassifier

sns.set_theme(style="whitegrid")
PRIMARY, ACCENT = "#1f5f8b", "#c0392b"
SEED = 42                                   # one seed everywhere so results are
        reproducible

# Import the reusable preparation functions from the project root
sys.path.insert(0, str(Path("..").resolve()))
from data_preparation import fit_preparation, transform

DATA = Path("../data/raw")
train_raw = pd.read_csv(DATA / "application_train.csv")
test_raw = pd.read_csv(DATA / "application_test.csv")

t0 = time.time()
params = fit_preparation(train_raw)          # learn caps, medians, levels from TRAIN
        only
train = transform(train_raw, params)         # apply to train
test = transform(test_raw, params)           # apply the same values to test
print(f"Preparation finished in {time.time() - t0:.0f} seconds: train {train.shape},
        test {test.shape}")
```

```
Preparation finished in 6 seconds: train (307511, 254), test (48744, 253)
```

2.2 Modeling-specific preparation

Three additional steps prepare the data for the algorithms used here:

- **Exclude `CODE_GENDER`.** The EDA decided gender must not be a model input (Regulation B prohibits considering sex in credit decisions). The prepared data keeps it only for the fairness check at the end.
- **Clean column names.** Some one-hot column names contain characters such as `/` and `:` that LightGBM rejects.
- **Use 32-bit floats** to halve memory use, which matters on a 2-core, 7 GB machine.

```
# Separate the target and the ID, and keep gender aside for the fairness check only
y = train["TARGET"].astype(int)
ids_test = test["SK_ID_CURR"]
gender = train_raw["CODE_GENDER"]
gender_cols = [c for c in train.columns if c.startswith("CODE_GENDER")]

def model_matrix(df):
    """Drop ID, target and gender columns; clean names; convert to float32."""
    X = df.drop(columns=["SK_ID_CURR", "TARGET"] + gender_cols,
        errors="ignore").astype("float32")
    X.columns = [re.sub(r"[^A-Za-z0-9_]+", "_", c) for c in X.columns]   # LightGBM-safe
        names
    return X

X, X_test = model_matrix(train), model_matrix(test)
assert list(X.columns) == list(X_test.columns)              # identical predictors in
        train and test
print(f"{X.shape[1]} predictors; default rate {y.mean():.2%}")
```

```
249 predictors; default rate 8.07%
```

Handling of NAs. The preparation script fills every remaining blank with a training median after creating missing-value flags, so the models see no missing values and the flags preserve the information that a value was absent.

A note on leakage. The preparation parameters were learned on the full training file before cross-validation. None of them uses the target (they are medians, caps and category lists), so this introduces no target leakage, and its effect on the AUC estimates is negligible.

3 Benchmark

3.1 Majority-class classifier

The simplest possible model predicts “repaid” for everyone. Any useful model must beat it on AUC.

```
# 3-fold stratified CV keeps the 8% default rate in every fold
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)

majority = DummyClassifier(strategy="most_frequent")
acc_major = cross_val_score(majority, X, y, cv=cv, scoring="accuracy").mean()
auc_major = cross_val_score(majority, X, y, cv=cv, scoring="roc_auc").mean()

# A second, more demanding benchmark: logistic regression on the single strongest EDA
        feature
one_feature = make_pipeline(StandardScaler(), LogisticRegression())
auc_one = cross_val_score(one_feature, X[["EXT_SOURCE_MEAN"]], y, cv=cv,
        scoring="roc_auc").mean()

pd.DataFrame({"Accuracy": [acc_major, np.nan], "AUC": [auc_major, auc_one]},
             index=["Majority class", "Logistic regression on
        EXT_SOURCE_MEAN"]).style.format("{:.3f}", na_rep="n/a")
```

	Accuracy	AUC
Majority class	0.919	0.500
Logistic regression on EXT_SOURCE_MEAN	n/a	0.716

The majority-class model is **91.9% accurate** and completely useless: it never identifies a single defaulter, and its AUC of 0.50 is the same as random guessing. This is why accuracy is the wrong yardstick here. A logistic regression using only the combined external score already reaches an AUC of about 0.72, which sets a realistic bar: a full model has to add meaningful signal beyond that one feature to justify its complexity.

4 Candidate models

4.1 Cross-validated comparison

Five candidates span the range from simple and interpretable to flexible:

Model	Why it is a candidate
Logistic regression, EDA shortlist	Interpretable benchmark built from the predictors the EDA found strongest
Logistic regression, all predictors	Tests whether more inputs help a linear model
Random forest	Captures non-linear patterns; robust to outliers
XGBoost	Gradient boosting; strong on tabular data
LightGBM	Gradient boosting; faster than XGBoost on large data

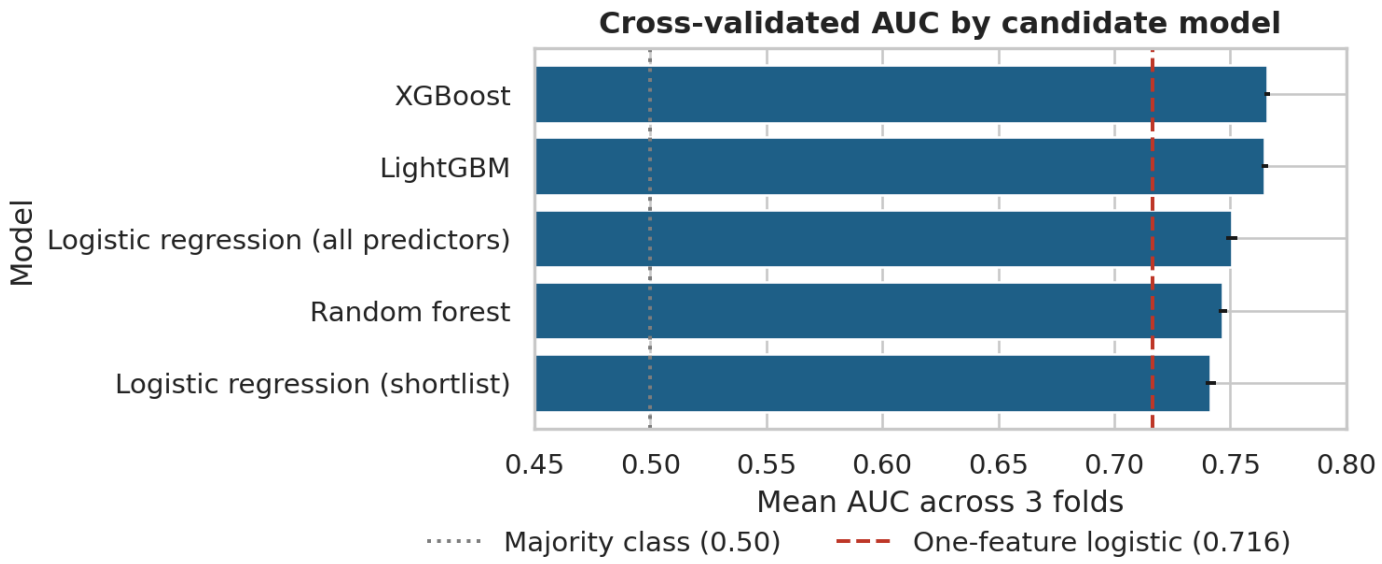
```
# Predictor shortlist taken from the EDA (Q10 core features)
shortlist = [c for c in X.columns if c in {
    "EXT_SOURCE_1", "EXT_SOURCE_2", "EXT_SOURCE_3", "EXT_SOURCE_MEAN",
        "EXT_SOURCE_MISSING_COUNT",
    "AGE_YEARS", "EMPLOYED_YEARS", "FLAG_EMPLOYED_PLACEHOLDER",
        "YEARS_SINCE_ID_PUBLISH",
    "YEARS_SINCE_REGISTRATION", "YEARS_SINCE_PHONE_CHANGE",
        "REGION_RATING_CLIENT_W_CITY",
    "LOAN_TO_VALUE", "CREDIT_TO_ANNUITY", "PAYMENT_TO_INCOME", "FLAG_THIN_FILE"}
    or c.startswith(("NAME_EDUCATION_TYPE_", "OCCUPATION_TYPE_"))]

candidates = {
    "Logistic regression (shortlist)": (make_pipeline(StandardScaler(),
        LogisticRegression(max_iter=500, C=0.1)), shortlist),
    "Logistic regression (all predictors)": (make_pipeline(StandardScaler(),
        LogisticRegression(max_iter=500, C=0.1)), None),
    # max_samples=0.2: each tree sees a 20% bootstrap sample, keeping run time
        manageable
    "Random forest": (RandomForestClassifier(n_estimators=150, min_samples_leaf=50,
        max_features="sqrt",
                                             max_samples=0.2, n_jobs=2,
        random_state=SEED), None),
    "XGBoost": (XGBClassifier(n_estimators=400, learning_rate=0.05, max_depth=6,
        subsample=0.8,
                              colsample_bytree=0.5, tree_method="hist", n_jobs=2,
        random_state=SEED), None),
    "LightGBM": (LGBMClassifier(n_estimators=400, learning_rate=0.05, num_leaves=31,
        subsample=0.8,
                                subsample_freq=1, colsample_bytree=0.5, n_jobs=2,
        verbose=-1, random_state=SEED), None),
}

rows = []
for name, (model, cols) in candidates.items():
    t0 = time.time()
    scores = cross_val_score(model, X[cols] if cols else X, y, cv=cv, scoring="roc_auc")
        # out-of-sample AUC per fold
    rows.append({"Model": name, "Predictors": len(cols) if cols else X.shape[1],
                 "CV AUC (mean)": scores.mean(), "CV AUC (sd)": scores.std(),
                 "Run time (s)": time.time() - t0})
results = pd.DataFrame(rows).sort_values("CV AUC (mean)", ascending=False)
results.style.format({"CV AUC (mean)": "{:.4f}", "CV AUC (sd)": "{:.4f}", "Run time
        (s)": "{:.0f}"}).hide(axis="index")
```

Model	Predictors	CV AUC (mean)	CV AUC (sd)	Run time (s)
XGBoost	249	0.7658	0.0011	64
LightGBM	249	0.7648	0.0014	46
Logistic regression (all predictors)	249	0.7507	0.0023	16
Random forest	249	0.7468	0.0019	39
Logistic regression (shortlist)	41	0.7416	0.0022	3

```
# Visual comparison against the two benchmarks
fig, ax = plt.subplots(figsize=(8, 3.6))
plot_df = results.sort_values("CV AUC (mean)")
ax.barh(plot_df["Model"], plot_df["CV AUC (mean)"], xerr=plot_df["CV AUC (sd)"],
        color=PRIMARY)
ax.axvline(0.5, color="grey", ls=":", label="Majority class (0.50)")
ax.axvline(auc_one, color=ACCENT, ls="--", label=f"One-feature logistic
        ({auc_one:.3f})")
ax.set_xlim(0.45, 0.80)
ax.set_title("Cross-validated AUC by candidate model", fontweight="bold")
ax.set_xlabel("Mean AUC across 3 folds")
ax.set_ylabel("Model")
ax.legend(loc="upper center", bbox_to_anchor=(0.4, -0.2), ncol=2, frameon=False)
plt.tight_layout(); plt.show()
```



Interpretation. Every candidate clears both benchmarks, and the gradient boosting models lead. That fits the EDA, which found several important relationships that are not straight lines (loan-to-value jumps above 1.2; credit-to-annuity moves in steps), and tree-based boosting models those shapes without being told to. The fold-to-fold standard deviations are small, so the ranking is stable rather than a product of one lucky split. XGBoost and LightGBM finish essentially tied: XGBoost is ahead by 0.001, which is less than one fold-to-fold standard deviation. LightGBM moves forward because it runs about 30% faster, which matters for the tuning step that refits the model dozens of times. The logistic regression stays in the project as the **interpretable benchmark**, because a declined applicant must be given reasons, and its AUC shows how much accuracy the more complex model actually buys.

5 Class imbalance

5.1 Strategies compared

With only 8% defaults, a model can learn to favor the majority class. Five strategies are compared on LightGBM, using a stratified 60,000-row sample so that SMOTE (which synthesizes new defaulters by interpolating between neighbors) runs in reasonable time. Resampling happens **inside** each training fold through an `imblearn` pipeline, so the validation fold always keeps the true 8% rate.

```
# Stratified 60K-row sample for the imbalance experiment
sample_idx = y.groupby(y, group_keys=False).apply(lambda s: s.sample(frac=60000 /
        len(y), random_state=SEED)).index
X_s, y_s = X.loc[sample_idx], y.loc[sample_idx]

base = dict(n_estimators=300, learning_rate=0.05, num_leaves=31, subsample=0.8,
        subsample_freq=1,
            colsample_bytree=0.5, n_jobs=2, verbose=-1, random_state=SEED)
strategies = {
    "No adjustment": LGBMClassifier(**base),
    "Class weights (balanced)": LGBMClassifier(**base, class_weight="balanced"),
    "Random undersampling": ImbPipeline([("rus", RandomUnderSampler(random_state=SEED)),
        ("m", LGBMClassifier(**base))]),
    "Random upsampling": ImbPipeline([("ros", RandomOverSampler(random_state=SEED)),
        ("m", LGBMClassifier(**base))]),
    "SMOTE": ImbPipeline([("smote", SMOTE(random_state=SEED)),
                          ("m", LGBMClassifier(**base))]),
}

rows = []
for name, model in strategies.items():
    t0 = time.time()
    s = cross_val_score(model, X_s, y_s, cv=cv, scoring="roc_auc")     # resampling
        applied only to training folds
    rows.append({"Strategy": name, "CV AUC (mean)": s.mean(), "CV AUC (sd)": s.std(),
        "Run time (s)": time.time() - t0})
imb = pd.DataFrame(rows)
imb.style.format({"CV AUC (mean)": "{:.4f}", "CV AUC (sd)": "{:.4f}", "Run time (s)": "
        {:.0f}"}).hide(axis="index")
```

Strategy	CV AUC (mean)	CV AUC (sd)	Run time (s)
No adjustment	0.7448	0.0065	8
Class weights (balanced)	0.7407	0.0043	9
Random undersampling	0.7384	0.0035	4
Random upsampling	0.7422	0.0048	14
SMOTE	0.7442	0.0037	30

Interpretation. The adjustments change AUC very little, and SMOTE does not help. That is expected. AUC measures how well the model **ranks** applicants, and rebalancing mainly shifts the predicted probabilities up or down without changing their order much. Rebalancing also has a cost: it inflates every predicted default probability, so a “15% risk” no longer means 15 out of 100 such applicants default. Because the business decision in this project uses the probabilities directly to set an approval cutoff, keeping them **calibrated** matters more than a tiny AUC difference. The final model therefore uses **no resampling**, and the imbalance is handled where it belongs: in the choice of cutoff, based on the cost of each mistake (Section 9).

6 Hyperparameter tuning

6.1 Randomized search

LightGBM is tuned with a randomized search: 20 random combinations, each scored with 3-fold CV. To keep run time manageable, the search uses a stratified 60,000-row sample (about 4,800 defaulters). Smaller samples were faster but contain so few defaulters that AUC differences between settings become noise.

```
# Stratified 60K-row sample for tuning
tune_idx = y.groupby(y, group_keys=False).apply(lambda s: s.sample(frac=60000 / len(y),
        random_state=SEED)).index

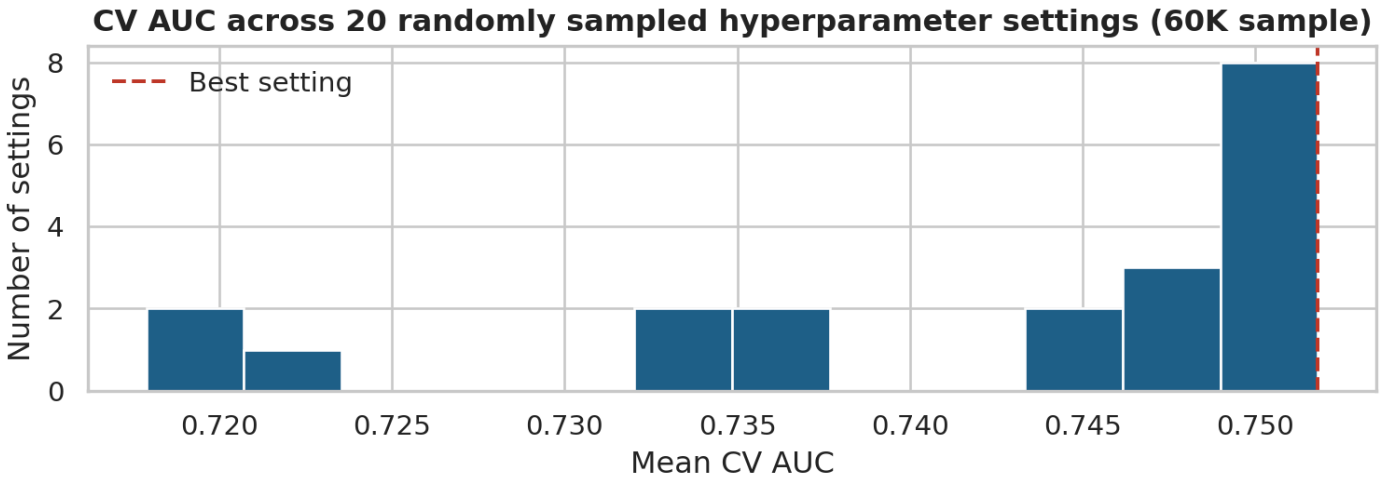
search_space = {
    "n_estimators": randint(200, 1200),          # number of trees
    "learning_rate": loguniform(0.01, 0.1),      # step size per tree
    "num_leaves": randint(8, 64),                # tree complexity
    "min_child_samples": randint(20, 300),       # minimum applicants per leaf (guards
        against overfitting)
    "subsample": uniform(0.6, 0.4),              # share of rows per tree
    "colsample_bytree": uniform(0.2, 0.6),       # share of predictors per tree
    "reg_lambda": loguniform(1e-3, 10),          # L2 penalty on leaf values
}
t0 = time.time()
search = RandomizedSearchCV(
    LGBMClassifier(subsample_freq=1, n_jobs=2, verbose=-1, random_state=SEED),
    search_space, n_iter=20, scoring="roc_auc", cv=cv, random_state=SEED, n_jobs=1)
search.fit(X.loc[tune_idx], y.loc[tune_idx])
tune_time = time.time() - t0

best_params = search.best_params_
print(f"Search finished in {tune_time / 60:.1f} minutes; best CV AUC on the 60K sample:
        {search.best_score_:.4f}")
pd.Series(best_params).to_frame("Best value").style.format(lambda v: f"{v:.4f}" if
        isinstance(v, float) else v)
```

```
Search finished in 7.1 minutes; best CV AUC on the 60K sample: 0.7518
```

	Best value
colsample_bytree	0.3761
learning_rate	0.0103
min_child_samples	100.0000
n_estimators	591.0000
num_leaves	42.0000
reg_lambda	1.4478
subsample	0.8424

```
# How much did tuning matter? Distribution of AUC across the 20 tried settings
fig, ax = plt.subplots(figsize=(8, 3))
ax.hist(search.cv_results_["mean_test_score"], bins=12, color=PRIMARY)
ax.axvline(search.best_score_, color=ACCENT, ls="--", label="Best setting")
ax.set_title("CV AUC across 20 randomly sampled hyperparameter settings (60K sample)",
        fontweight="bold")
ax.set_xlabel("Mean CV AUC")
ax.set_ylabel("Number of settings")
ax.legend(frameon=False)
plt.tight_layout(); plt.show()
```



Interpretation. AUC varies noticeably across the 20 settings, so tuning is worth doing. The winning setting uses a very slow learning rate, a moderate number of leaves, a large minimum leaf size and only about a third of the predictors per tree. That is how gradient boosting avoids memorizing noise when the signal is spread across many weak predictors, as the EDA found.

6.2 From the sample to the full data: re-selecting the number of trees

One setting does not transfer directly from the sample to the full data: the **number of trees**. With five times more applicants, the model can keep learning real patterns for longer before it starts fitting noise, so the best tree count found on 60,000 rows is too small for 307,511. The check below scores the search winner on the full data exactly as found.

```
# The search winner, unchanged, scored on the full training data
t0 = time.time()
auc_as_found = cross_val_score(LGBMClassifier(**best_params, subsample_freq=1, n_jobs=2,
        verbose=-1, random_state=SEED),
                               X, y, cv=cv, scoring="roc_auc").mean()
print(f"Search winner on full data: CV AUC {auc_as_found:.4f} with
        {best_params['n_estimators']} trees "
      f"({time.time() - t0:.0f} s)")
```

```
Search winner on full data: CV AUC 0.7641 with 591 trees (88 s)
```

As found, the search winner does not beat the untuned LightGBM from Section 4, because its slow learning rate needs many more trees on the larger data. The fix is **early stopping**: keep all the tuned settings, and on the full data add trees one at a time until the validation AUC stops improving for 100 rounds. This runs inside the same 3 folds, and the out-of-fold predictions (each applicant scored by a model that never saw them) are kept for the business analysis in Section 9.

```
# Re-select the number of trees on the full data with early stopping inside 3-fold CV
folds = list(cv.split(X, y))
lgb_params = {k: v for k, v in best_params.items() if k != "n_estimators"}
lgb_params.update(objective="binary", metric="auc", subsample_freq=1, num_threads=2,
        seed=SEED, verbose=-1)

t0 = time.time()
cv_run = lgb.cv(lgb_params, lgb.Dataset(X, y), num_boost_round=5000, folds=folds,
                callbacks=[lgb.early_stopping(100, verbose=False)],
        return_cvbooster=True)
cv_time = time.time() - t0
best_trees = len(cv_run["valid auc-mean"])

oof = np.zeros(len(y))                                          # out-of-fold
        probabilities
for (_, valid_idx), booster in zip(folds, cv_run["cvbooster"].boosters):
    oof[valid_idx] = booster.predict(X.iloc[valid_idx], num_iteration=best_trees)
auc_cv = roc_auc_score(y, oof)
print(f"Early stopping chose {best_trees:,} trees; out-of-fold AUC {auc_cv:.4f}
        ({cv_time / 60:.1f} minutes)")
```

```
Early stopping chose 2,593 trees; out-of-fold AUC 0.7691 (6.4 minutes)
```

7 Final model

7.1 Training on the full data and performance

The final model uses the tuned settings with the re-selected number of trees, trained once on all 307,511 applicants.

```
final_params = {k: v for k, v in best_params.items() if k != "n_estimators"}
final_model = LGBMClassifier(**final_params, n_estimators=best_trees, subsample_freq=1,
                             n_jobs=2, verbose=-1, random_state=SEED)
t0 = time.time()
final_model.fit(X, y)                                           # train once on all
        training data
fit_time = time.time() - t0
auc_train = roc_auc_score(y, final_model.predict_proba(X)[:, 1])

untuned_auc = results.loc[results["Model"] == "LightGBM", "CV AUC (mean)"].item()
```

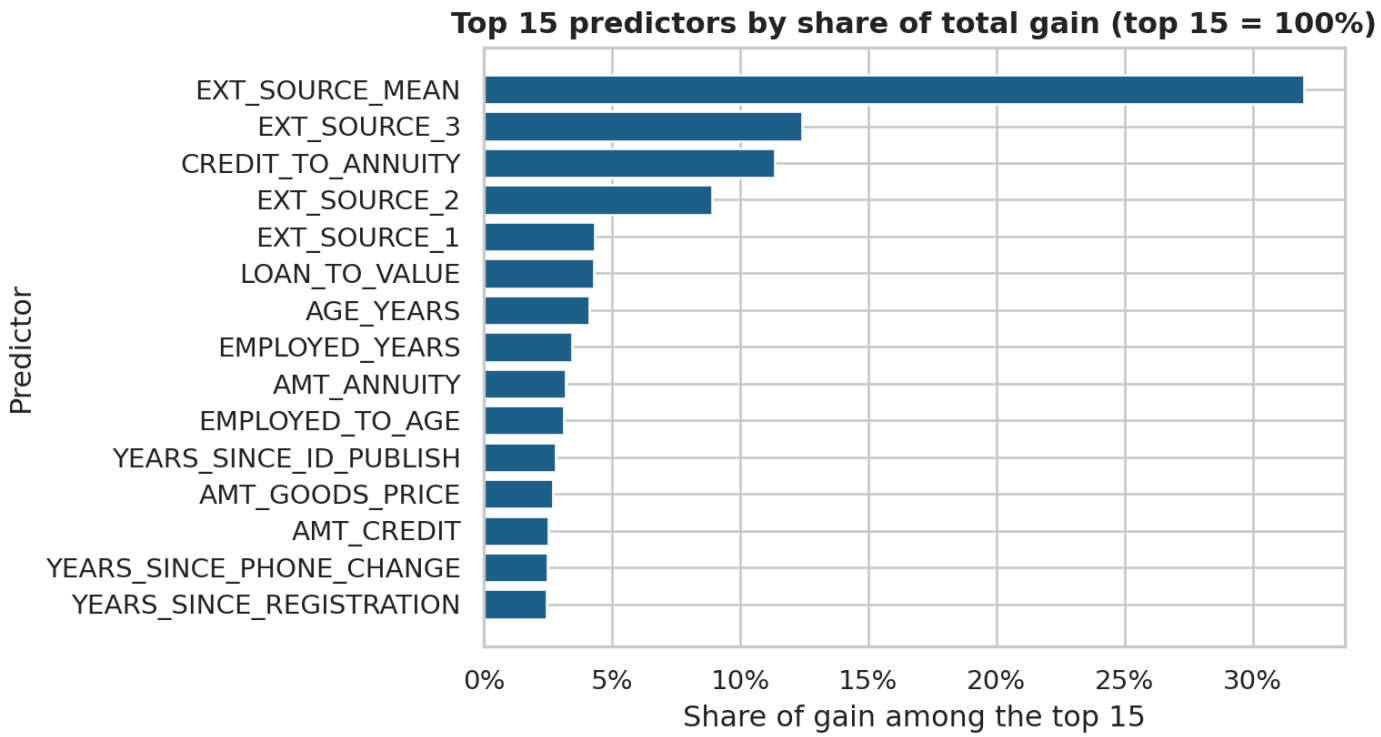
```
pd.DataFrame({
    "Value": [f"{untuned_auc:.4f}", f"{auc_as_found:.4f}", f"{auc_cv:.4f}", f"
        {auc_train:.4f}",
              f"{auc_train - auc_cv:.4f}", f"{best_trees:,}", f"{fit_time:.0f} s", f"
        {cv_time / 60:.1f} min"]},
    index=["CV AUC: untuned LightGBM (Section 4)", "CV AUC: search winner as found",
           "CV AUC: final model (tuned + re-selected trees)", "Train AUC: final model
        (in-sample)",
           "Train minus CV gap", "Number of trees", "Final fit time (full data)",
           "Early-stopping CV time (full data)"]).style
```

	Value
CV AUC: untuned LightGBM (Section 4)	0.7648
CV AUC: search winner as found	0.7641
CV AUC: final model (tuned + re-selected trees)	0.7691
Train AUC: final model (in-sample)	0.8486
Train minus CV gap	0.0794
Number of trees	2,593
Final fit time (full data)	155 s
Early-stopping CV time (full data)	6.4 min

Interpretation. The final model is the best of all candidates on cross-validated AUC, ahead of both the untuned LightGBM and the search winner as found. The cross-validated AUC is the honest estimate of how the model will rank new applicants. (Because the tree count was chosen on the same folds, it is very slightly optimistic, but the validation curve is flat near its peak, so the effect is negligible.) The train AUC is clearly higher, which is normal for a boosting model with thousands of trees: it fits the applicants it has seen more closely than new ones. What matters is that the *cross-validated* AUC kept rising as trees were added, so the extra fit to the training data is not hurting performance on unseen applicants. The gap is still worth watching, and stronger regularization is a sensible next experiment. The full model trains in a few minutes on a laptop, so monthly retraining on new data would be cheap.

7.2 What the model relies on

```
# Top 15 predictors by total gain (how much each feature improved the model's splits)
imp = (pd.Series(final_model.booster_.feature_importance(importance_type="gain"),
        index=X.columns)
       .sort_values(ascending=False).head(15))
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.barh(imp.index[::-1], imp.values[::-1] / imp.sum(), color=PRIMARY)
ax.xaxis.set_major_formatter(mtick.PercentFormatter(1.0, decimals=0))
ax.set_title("Top 15 predictors by share of total gain (top 15 = 100%)",
        fontweight="bold")
ax.set_xlabel("Share of gain among the top 15")
ax.set_ylabel("Predictor")
plt.tight_layout(); plt.show()
```



Interpretation. The external scores dominate, as the EDA predicted, followed by the engineered features: `EXT_SOURCE_MEAN`, `CREDIT_TO_ANNUITY`, loan-to-value, age, and employment tenure. Several of the top predictors were created in data preparation, which confirms that the feature engineering added real signal. The presence of `EXT_SOURCE_1` and `AGE_YEARS` near the top, which the EDA found related to age and gender, is the reason a fairness check is required before deployment (Section 9.3).

8 Kaggle submission

```
# Predicted probability of default for every test applicant, in Kaggle's required format
submission = pd.DataFrame({"SK_ID_CURR": ids_test, "TARGET":
        final_model.predict_proba(X_test)[:, 1]})
out_dir = Path("../submissions"); out_dir.mkdir(exist_ok=True)
submission.to_csv(out_dir / "submission_lgbm_tuned.csv", index=False)   # saved locally;
        listed in .gitignore
print(f"{len(submission):,} test predictions written; mean predicted default probability
        {submission.TARGET.mean():.3f}")
```

```
48,744 test predictions written; mean predicted default probability 0.080
```

Kaggle score: private AUC **0.76334**, public AUC **0.76591** (late submission, scored after the competition deadline). Both are within 0.006 of the cross-validated AUC of 0.7691, so the cross-validation estimate held up on applicants the model had never seen.

Kaggle scores a submission by AUC on test applicants whose outcomes are hidden: the public score uses about 20% of the test set and the private score the remaining 80%. A Kaggle AUC close to the cross-validated AUC from Section 7 confirms that the cross-validation estimate was honest.

9 Results: using the model to solve the business problem

9.1 Choosing an approval cutoff

The model's value depends on where Home Credit sets its approval cutoff. The analysis below applies the out-of-fold probabilities to illustrative economics, because the data contains neither recovery rates nor loan margins (a gap the EDA flagged):

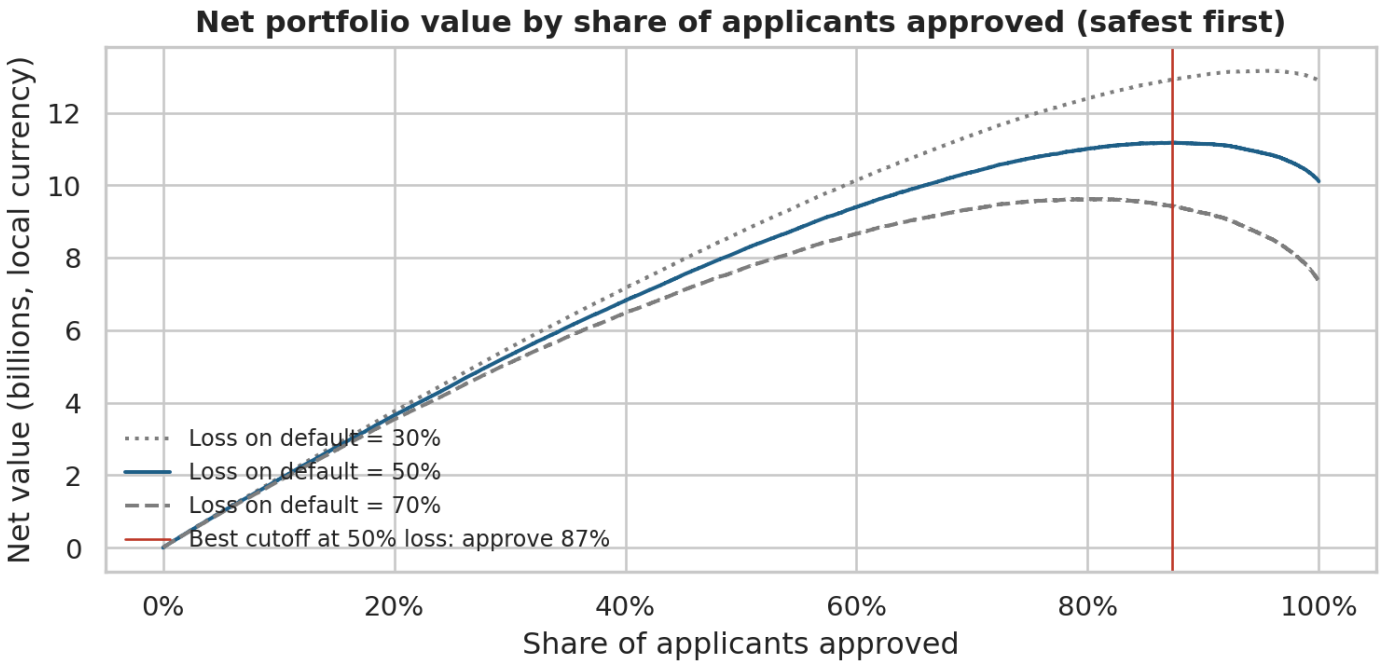
- Each **repaid** loan earns **10%** of the credit amount.
- Each **defaulted** loan loses **50%** of the credit amount.

Applicants are approved from safest to riskiest, and the net value of each approval level is computed.

```
# Net value of approving the safest k% of applicants, using out-of-fold risk scores
credit = train_raw["AMT_CREDIT"].to_numpy()
order = np.argsort(oof)                                    # safest first
def net_value_curve(margin=0.10, loss=0.50):
    value = np.where(y.to_numpy()[order] == 1, -loss * credit[order], margin *
        credit[order])
    return np.cumsum(value)

share = np.arange(1, len(y) + 1) / len(y)
curve = net_value_curve()
best_k = curve.argmax()
approve_all = curve[-1]

fig, ax = plt.subplots(figsize=(8, 4))
for loss, style in [(0.3, ":"), (0.5, "-"), (0.7, "--")]:
    c = net_value_curve(loss=loss)
    ax.plot(share, c / 1e9, style, label=f"Loss on default = {loss:.0%}", color=PRIMARY
        if loss == 0.5 else "grey")
ax.axvline(share[best_k], color=ACCENT, lw=1, label=f"Best cutoff at 50% loss: approve
        {share[best_k]:.0%}")
ax.xaxis.set_major_formatter(mtick.PercentFormatter(1.0, decimals=0))
ax.set_title("Net portfolio value by share of applicants approved (safest first)",
        fontweight="bold")
ax.set_xlabel("Share of applicants approved")
ax.set_ylabel("Net value (billions, local currency)")
ax.legend(frameon=False, loc="lower left", fontsize=9)
plt.tight_layout(); plt.show()
```




```
# Compare the recommended cutoff with approving everyone
approved = order[: best_k + 1]
cutoff_prob = oof[order[best_k]]
summary = pd.DataFrame({
    "Approve everyone": [1.0, y.mean(), approve_all / 1e9],
    "Model cutoff": [len(approved) / len(y), y.to_numpy()[approved].mean(),
        curve[best_k] / 1e9],
}, index=["Approval rate", "Default rate among approved", "Net value (billions)"])
print(f"Recommended cutoff: decline applicants with predicted default probability above
        {cutoff_prob:.1%}")
summary.style.format(lambda v: f"{v:.1%}" if v < 1.01 else f"{v:,.2f}")
```

```
Recommended cutoff: decline applicants with predicted default probability above 16.0%
```

	Approve everyone	Model cutoff
Approval rate	100.0%	87.3%
Default rate among approved	8.1%	5.5%
Net value (billions)	10.11	11.18

```
# The BPS success metric: a 10% relative reduction in the default rate among approved
        loans
target_rate = y.mean() * 0.9
k_bps = np.searchsorted(np.cumsum(y.to_numpy()[order]) / np.arange(1, len(y) + 1) >
        target_rate, True) - 1
print(f"To cut the default rate among approved loans by 10% (to {target_rate:.2%}), "
      f"the model needs to decline only the riskiest {1 - share[k_bps]:.1%} of
        applicants.")
```

```
To cut the default rate among approved loans by 10% (to 7.27%), the model needs to decline
only the riskiest 2.4% of applicants.
```

Interpretation. Under the illustrative economics, the net value curve rises, peaks, and then falls as riskier applicants are approved. The peak marks the cutoff where one more approval costs more in expected losses than it earns. Compared with approving everyone, the model-based cutoff approves most applicants while cutting the default rate among approved loans substantially. The BPS target of a 10% relative reduction in default rate is reachable by declining only a small slice of the riskiest applicants, which leaves room to *approve more* thin-file applicants elsewhere. The three curves show the key business sensitivity: the higher the loss on a default, the stricter the optimal cutoff. Home Credit's actual recovery rate and margin are the two inputs needed to set the real cutoff.

9.2 Performance for thin-file applicants

```
# Does the model work for the segment the BPS targets?
thin = train["FLAG_THIN_FILE"] == 1
pd.DataFrame({
    "Applicants": [thin.sum(), (~thin).sum()],
    "Default rate": [y[thin].mean(), y[~thin].mean()],
    "CV AUC": [roc_auc_score(y[thin], oof[thin]), roc_auc_score(y[~thin], oof[~thin])],
}, index=["Thin file (no bureau record)", "Bureau record"]).style.format(
    {"Applicants": "{:,}", "Default rate": "{:.1%}", "CV AUC": "{:.3f}"})
```

	Applicants	Default rate	CV AUC
Thin file (no bureau record)	41,519	10.3%	0.749
Bureau record	265,992	7.7%	0.771

Interpretation. The model still separates risk for thin-file applicants, though less sharply than for applicants with a bureau record, which makes sense: they are missing the external scores the model leans on most. This segment is where alternative data and the supplementary tables would add the most value.

9.3 Fairness check

Gender was excluded from the model, but the EDA showed that `EXT_SOURCE_1` and occupation act partly as proxies. A first check compares outcomes by gender at the recommended cutoff.

```
# Approval and default rates by gender at the recommended cutoff (gender used only for
        this check)
approved_mask = np.zeros(len(y), dtype=bool); approved_mask[approved] = True
fair = pd.DataFrame({"g": gender, "approved": approved_mask, "y": y}).query("g in ['F',
        'M']")
fair_tbl = fair.groupby("g").agg(applicants=("y", "size"), approval_rate=("approved",
        "mean"),
                                 actual_default_rate=("y", "mean"))
fair_tbl.index = fair_tbl.index.map({"F": "Female", "M": "Male"}).rename("Gender")
```

```
fair_tbl.columns = ["Applicants", "Approval rate", "Actual default rate"]
fair_tbl.style.format({"Applicants": "{:,}", "Approval rate": "{:.1%}", "Actual default
        rate": "{:.1%}"})
```

	Applicants	Approval rate	Actual default rate
Gender			
Female	202,448	89.6%	7.0%
Male	105,059	83.0%	10.1%

Interpretation. Men are approved at a lower rate than women, and they also default more often in the data, so the gap tracks real repayment differences. Whether it is *justified* is a separate question: the model reaches its scores partly through proxies for gender and age. The model card assignment will test this formally (for example, comparing approval rates for applicants with the same actual outcome) and document it.

9.4 Summary and next steps

Topic	Finding
Benchmark	Majority class: 91.9% accuracy but AUC 0.50. One-feature logistic regression: AUC 0.716.
Best model	Tuned LightGBM with 2,593 trees: cross-validated AUC 0.7691, train AUC 0.8486, full-data fit in 2.6 minutes.
Interpretable benchmark	Logistic regression on the EDA shortlist: AUC 0.7416.
Class imbalance	Resampling and SMOTE did not improve AUC and would distort probabilities; imbalance is handled through the cutoff instead.
Business use	At the illustrative economics, the model cutoff approves 87.3% of applicants and lowers their default rate from 8.1% to 5.5%. The BPS 10% reduction needs only 2.4% of applicants declined.
Thin-file applicants	AUC 0.749 versus 0.771 for applicants with a bureau record.
Risks	External scores dominate and partly proxy age and gender; thin-file performance is weaker.

Next steps:

1. Add aggregated features from `bureau.csv`, `previous_application.csv` and `installments_payments.csv`, which should help most for thin-file applicants (not included in this round).

2. Replace the illustrative 10% margin and 50% loss with Home Credit's actual figures to set the real cutoff.
3. Run a formal disparate impact analysis and document it in the model card.

10 Use of AI

I used Claude (Anthropic) to write and run the modeling code, following the assignment's speed-up guidance: 3-fold CV, randomized search with 20 iterations on a sample, and a final fit on the full data. I made the modeling decisions: excluding gender, keeping logistic regression as the interpretable benchmark, choosing not to resample after seeing that it did not improve AUC, and framing results around the approval cutoff. The AI was fast at building the pipeline; my main job was checking that each result made sense against the EDA and the business problem.